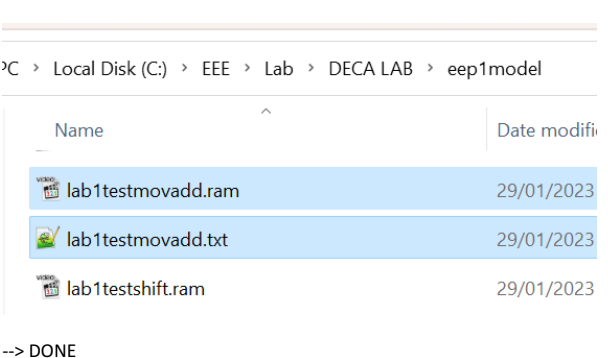
DECA 3.1 - EEP1 Datapath & ALU

Sunday, January 29, 2023 12:32 AM

Task 1 – Download eep1 model and lab1 Issie projects, and update Issie

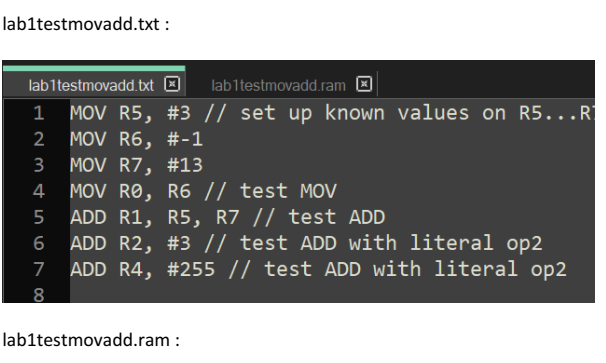
- Step 1 : copy lab1testmovadd.txt and lab1testmovadd.ram files from lab1 to eep1model


PC › Local Disk (C:) › EEE › Lab › DECA LAB › eep1model
Name
Date modifi
lab1testmovadd.ram 29/01/2023
lab1testmovadd.txt 29/01/2023
lab1testshift.ram 29/01/2023

--> DONE

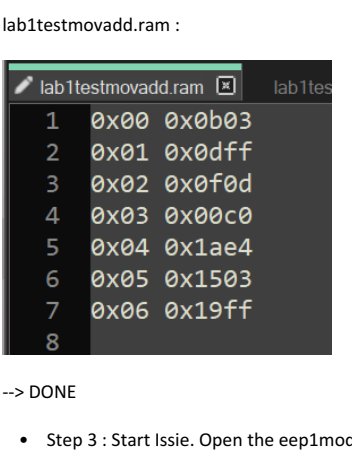
- Step 2 : open lab1testmovadd.ram, and lab1testmovadd.txt

lab1testmovadd.txt :



```
MOV R5, #3 // set up known values on R5...R7
MOV R6, #-1
MOV R7, #13
MOV R0, R6 // test MOV
ADD R1, R5, R7 // test ADD
ADD R2, #3 // test ADD with literal op2
ADD R4, #255 // test ADD with literal op2
```

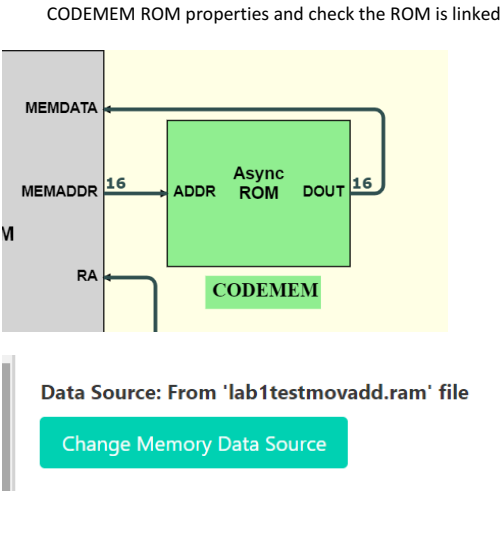
lab1testmovadd.ram :



```
0x00 0x0b03
0x01 0x0dff
0x02 0x0f0d
0x03 0x00c0
0x04 0x1ae4
0x05 0x1503
0x06 0x19ff
```

--> DONE

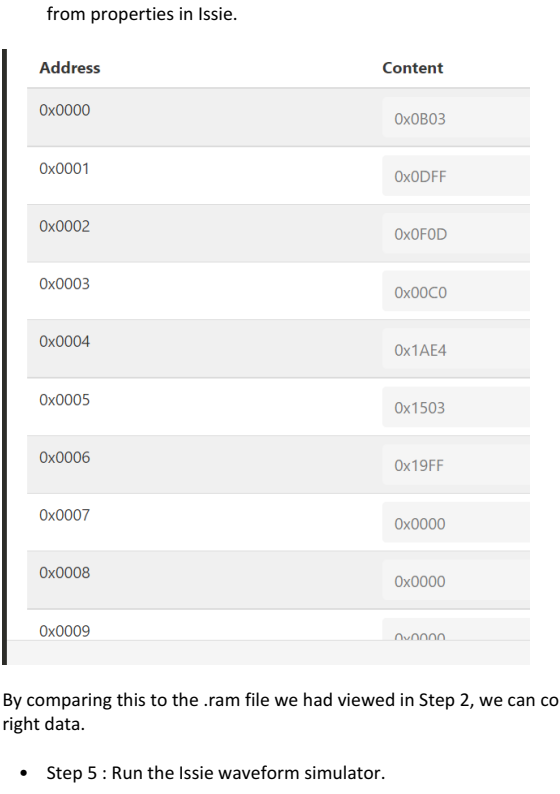
- Step 3 : Start Issie. Open the eep1model project you downloaded previously. Open the eep1model sheet. Select the CODEMEM ROM properties and check the ROM is linked to lab1testmovadd.ram.



Data Source: From 'lab1testmovadd.ram' file

--> DONE

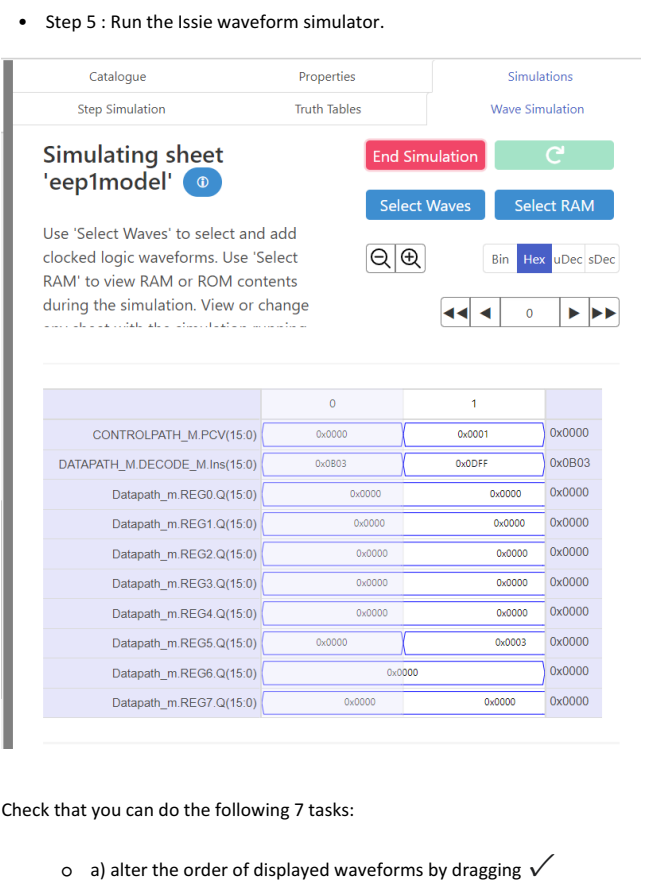
- Step 4 : Check that the CODEMEM ROM has the same data shown in the machine code file by opening up the ROM viewer from properties in Issie.



Address	Content
0x0000	0x0B03
0x0001	0x0DFF
0x0002	0x0F0D
0x0003	0x00C0
0x0004	0x1AE4
0x0005	0x1503
0x0006	0x19FF
0x0007	0x0000
0x0008	0x0000
0x0009	0x0000

By comparing this to the .ram file we had viewed in Step 2, we can confirm that the CODEMEM component does indeed have the right data.

- Step 5 : Run the Issie waveform simulator.

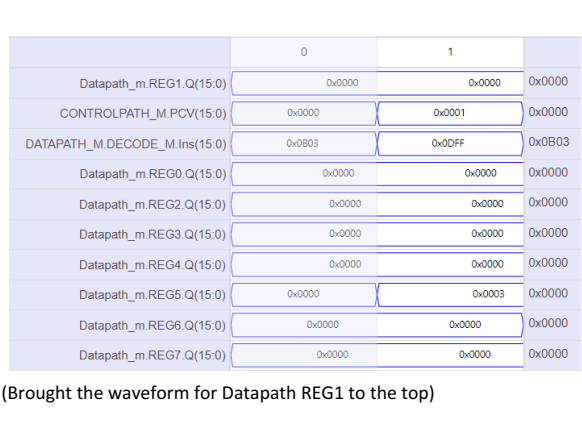


Simulating sheet 'eep1model'

Use 'Select Waves' to select and add clocked logic waveforms. Use 'Select RAM' to view RAM or ROM contents during the simulation. View or change

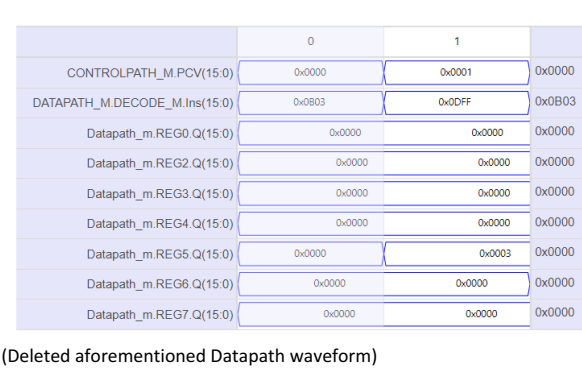
Check that you can do the following 7 tasks:

- a) alter the order of displayed waveforms by dragging ✓



(Brought the waveform for Datapath REG1 to the top)

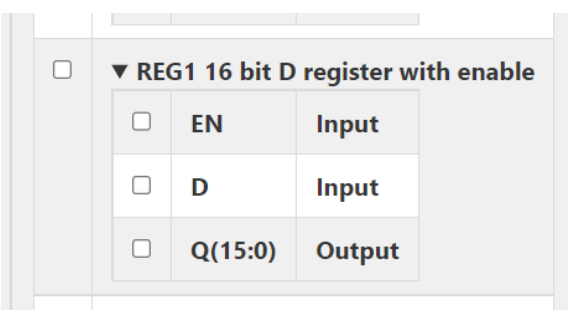
- b) delete a displayed waveform ✓



(Deleted aforementioned Datapath waveform)

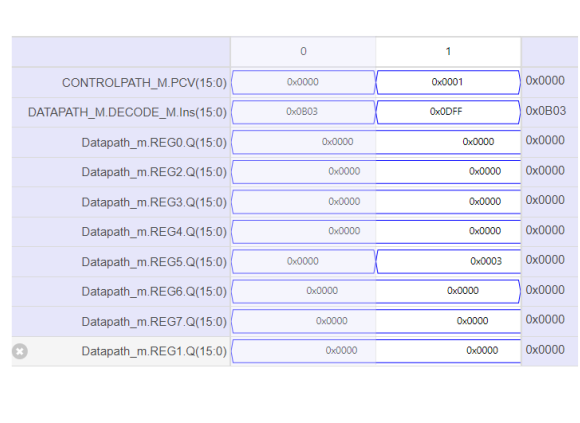
- c) add a new displayed waveform by searching for part of its name ✓

After searching for Datapath:

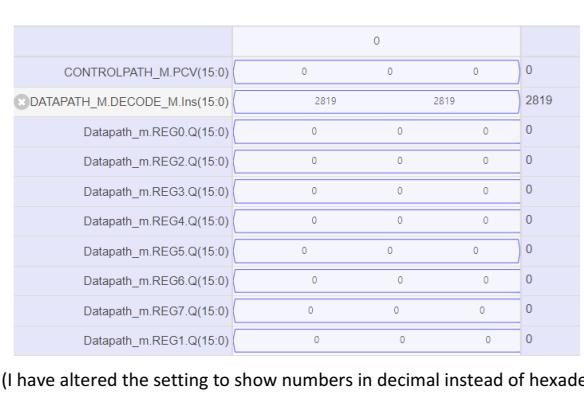


☐	▼ REG1 16 bit D register with enable
☐ EN	Input
☐ D	Input
☐ Q(15:0)	Output

After checking the box adjacent to Q(15:0) Output:



- d) change the radix of displayed waveforms ✓

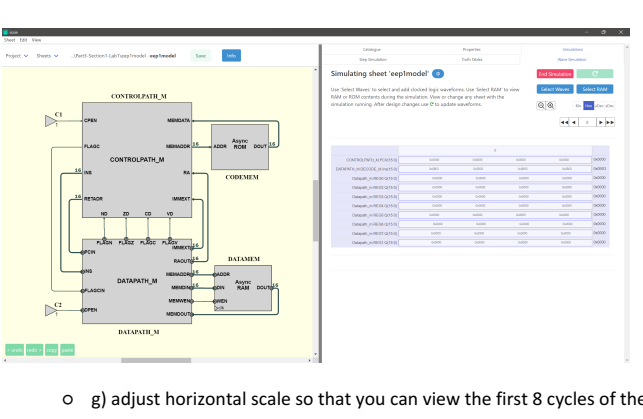


(I have altered the setting to show numbers in decimal instead of hexadecimal)

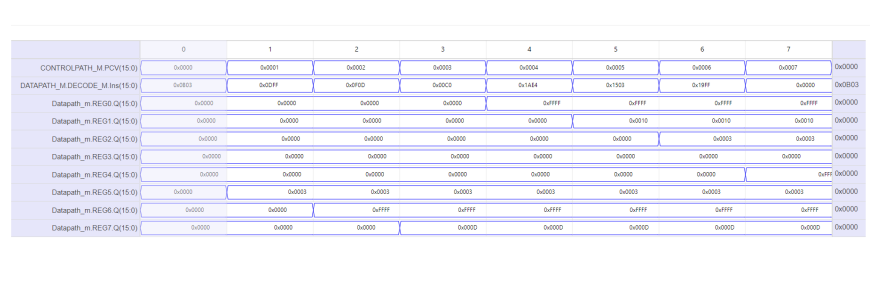
- e) change the position of the cursor by clicking waveforms

I am unsure what this means.

- f) adjust the grey vertical divider to make the simulation tab larger ✓



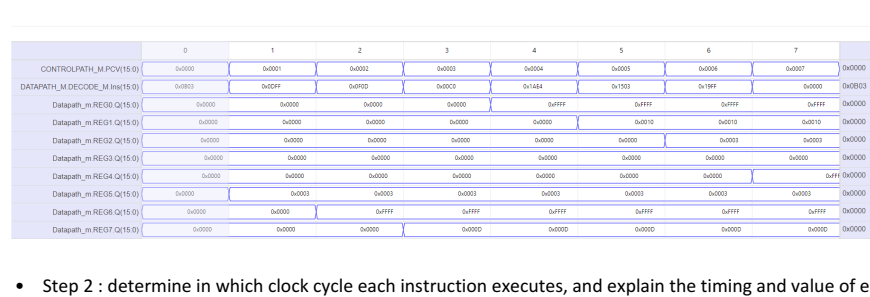
- g) adjust horizontal scale so that you can view the first 8 cycles of the CPU operation while also seeing the value of the PC (PCV) displayed in hex ✓



Task 1 complete.

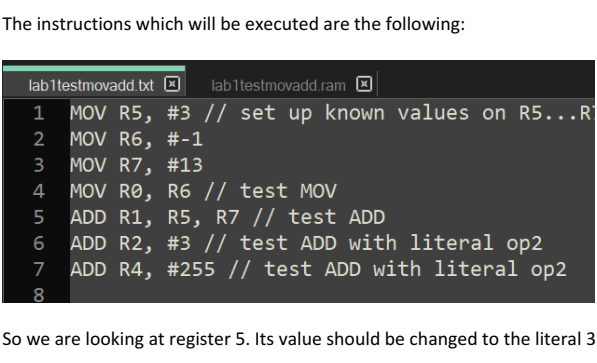
Task 2 - Use Issie's waveform simulator to view the first 8 clock cycles of sheet eep1model simulation using the lab1testmovadd.txt instructions. Determine in which clock cycle each instruction executes, and explain the timing and value of each Datapath register Q waveform change using the instructions in lab1text1.txt and the ALU instruction reference in Figure 10.

- Step 1 : use waveform simulator to view the first 8 clocks of the eep1model simulation



- Step 2 : determine in which clock cycle each instruction executes, and explain the timing and value of each Datapath register Q waveform change using the instructions in lab1text1.txt and the ALU instruction reference in Figure 10

The instructions which will be executed are the following:

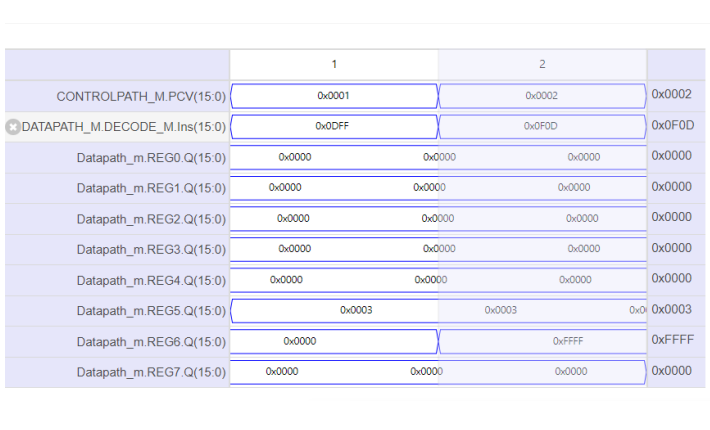


```
MOV R5, #3 // set up known values on R5...R7
MOV R6, #-1
MOV R7, #13
MOV R0, R6 // test MOV
ADD R1, R5, R7 // test ADD
ADD R2, #3 // test ADD with literal op2
ADD R4, #255 // test ADD with literal op2
```

So we are looking at register 5. Its value should be changed to the literal 3 $3_{10} = 0x003_{16}$

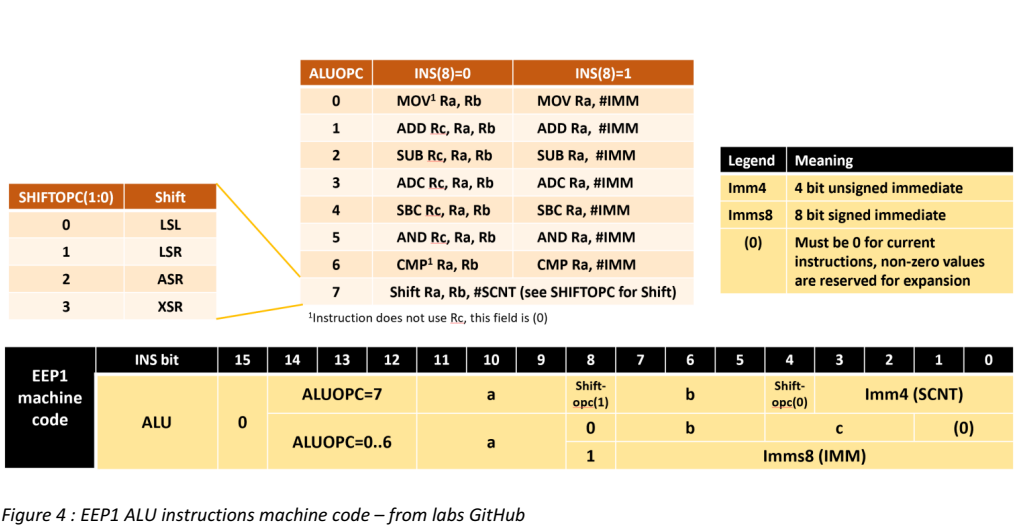
On the waveforms, we can see that the first instruction **MOV R5, #3** is executed at clock 1, so by PC = 1, R5 = 3 (as we can see on the waveform for Datapath Register 5, when ControlPath = 0x0001, R5 = 0x0003).

The second instruction is **MOV R6, #-1**. This instruction is executed at clock 2, the register R6 has been set to literal –1 (as we can see on the waveform, at clock 2, REG6 = 0xffff, which is –1 in 2s complement in base 16).



Task 2 complete.

Task 3 - For MOV and ADD instructions with literal and register op2 (4 cases) choose an example from lab1testmovadd.txt and use Figure 4 to calculate the corresponding machine code in binary, then convert it to hexadecimal. Check that your work agrees with lab1testmovadd.ram.



ALUOPC	INS(8)=0	INS(8)=1
0	MOV[1] Ra, Rb	MOV Ra, #IMM
1	ADD Rc, Ra, Rb	ADD Ra, #IMM
2	SUB Rc, Ra, Rb	SUB Ra, #IMM
3	ADC Rc, Ra, Rb	ADC Ra, #IMM
4	SBC Rc, Ra, Rb	SBC Ra, #IMM
5	AND Rc, Ra, Rb	AND Ra, #IMM
6	CMP[1] Ra, Rb	CMP Ra, #IMM
7	Shift Ra, Rb, #SCNT (see SHIFTOPC for Shift)	

[1] Instruction does not use Rc, this field is (0)

SHIFTOPC(1:0)	Shift
0	LSL
1	LSR
2	ASR
3	XSR

Legend	Meaning
Imm4	4 bit unsigned immediate
Imms8	8 bit signed immediate
(0)	Must be 0 for current instructions, non-zero values are reserved for expansion

	INS bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EEP1 machine code	ALU	0	ALUOPC=7			a			Shift-opc(1)	b			Shift-opc(0)	Imm4 (SCNT)			
			ALUOPC=0..6			a			0	b			c			(0)	
									1	Imms8 (IMM)							

Figure 4 : EEP1 ALU instructions machine code – from labs GitHub

Let's take instruction 3 from lab1testmovadd.txt : **MOV R7, #13.**

```
A = 13

Int a = 13;

MOV R7, #13
STR R7 [0xFFFF]

0 000 111 1 0000 1101
```

The number 13 in decimal is equal to 0x000d in hexadecimal.
In signed binary, it is equal to 1101. In this instance, we need an 8-bit signed immediate, therefore we will take 00001101 instead (same number, but 4 0s added to MSB to get 8-bit value).

We can determine from Figure 4 that, since our instruction consists of **MOV Ra, #IMM**, INS(8)=1 (header of leftmost column) ; and that ALUOPC = 0 (first row).
Figure 4 gives us machine code for Ra. In our case, a = 6, since we are setting Register 6 to the value 13.

0 as per ... because this instruction is MOV. $O_p = 000_2$ · $a = 7_{10} = 111_2$ · because operating on an immediate · binary (8 bit immediate) · registers

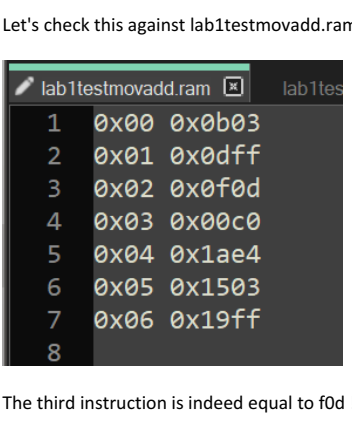
We can therefore find the machine code for this instruction:

0 000 111 1 00001101

This gives us the binary number 0000111100001101, which when converted to hexadecimal, is equal to f0d.

Now that the combinational logic has been done, it is time to test whether it works as intended. The instructions for Task 5 tell us to test the combinational logic for MOV and ADD instructions. In order to do this,

Let's check this against lab1testmovadd.ram :



```
0x00 0x0b03
0x01 0x0dff
0x02 0x0f0d
0x03 0x00c0
0x04 0x1ae4
0x05 0x1503
0x06 0x19ff
```

The third instruction is indeed equal to f0d !

Task 3 complete.

Task 4 - Write a 4 row algebraic truth table for Regfile.Din1 with algebraic inputs A, B, C, Imm8, and binary inputs Op2Sel, AD1SelC (assuming that ALUOPC is fixed as from Figure 4 is required to implement the MOV instruction).
Determine which row makes the correct dataflow for each of the two forms of MOV instruction (literal and register op2).
Write a similar truth table with ALUOPC fixed as for ADD, and determine which row implements each of the two forms of the ADD instruction.
Using these results, and the MOV and ADD instruction coding from Figure 4, write a truth table with inputs ALUOPC, INS(8), and outputs Op2Sel, AD1SelC, which correctly implements these outputs for MOV and ADD instructions.
You do not yet need to include rows in the truth table for ALUOPC values other than those for MOV and ADD.

"Create a 4-row algebraic truth table for Regfile.Din1 with inputs A, B, C, Imm8, Op2Sel, AD1SelC, and fixed ALUOPC for the MOV instruction. Identify the rows that correspond to the two forms of the MOV instruction (literal and register op2) and repeat the process for the ADD instruction. Finally, create a truth table with inputs ALUOPC, INS(8), and outputs Op2Sel, AD1Sel that implements the MOV and ADD instructions."

Block	Select	Use
Datapath.MUX1	AD1SelC	Determines REGFILE Port 2 address
Datapath.MUX2	0	Assume 0
Datapath.MUX3	0	Assume 0
ALU.MUX1	OP2Sel	Selects op2 format
ALU.MUX2	ALUOPC	Selects which ALU operation
ALU.ADDSUB	/	/
EXTEND	/	Assume EXTEND.IMMEXT = EXTEND.IMMS8 sign extended to 16 bits
Datapath.REGFILE	/	Outputs Qn for ADn

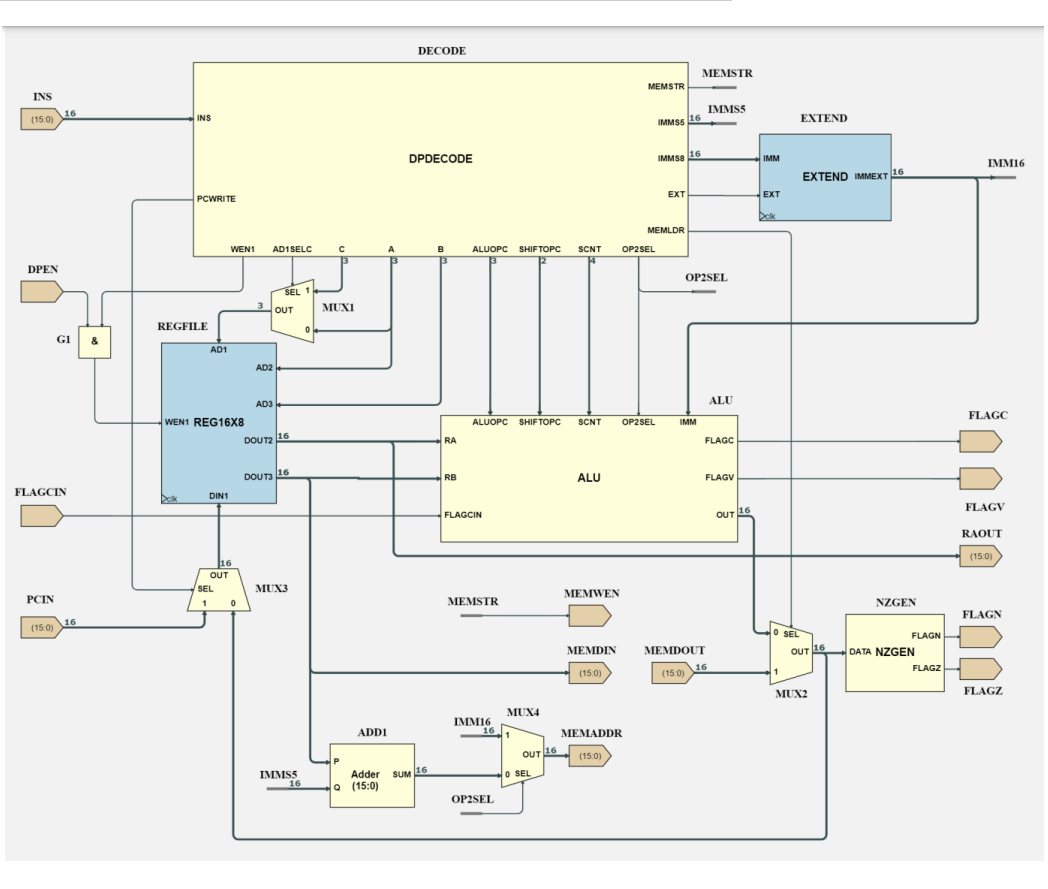


Figure 7: EEP1 Datapath – from labs GitHub

A	B	C	Imm8	Op2Sel	AD1SelC	ALUOPC	Regfile.Din1
a	b	X	X	0	0	0	Ra := Rb
X	b	c	X	0	1	0	Rc := Rb
a	X	X	k	1	0	0	Ra := k
X	X	c	k	1	1	0	Rc := k

*Dataflow truth table for **MOV** instruction*

ALUOPC set to constant 0 because we are only using instruction **MOV** (see Figure 4). Additionally, Ins(8) should be set to 1 if we are setting the register to an immediate value, and should be set to 0 if we are setting the register to the value of another register.

Question: What is AD1SelC ?
Answer: AD1SelC is the select line that determines whether A or C will be passed through to Regfile.AD1. If this variable is 1, then C is passed through (hence the Sel C in its name), and if it is a 0, then A is passed through.
Remark: Now that I understand the functionality, the name is much more meaningful. Prior, it was rather enigmatic.

Question: What are algebraic inputs A, B and C ?
Answer: a is the number of the first register (as in Ra). B is the value of the second register (as in Rb). All 3 of these inputs are 3 bits wide. I do not know what the C input is. Intuitively, I would guess that it is to specify the 3rd register R_C, but that is not being used here. The **instructions which involve R_C are invalid**.
--> "Determine which row makes the correct dataflow for each of the two forms of MOV instruction (literal and register op2)", from the instructions for Task 4. Since the two instructions involving R_C are invalid, the two rows with **correct dataflows are rows 1 and 3**.

Question: What does Op2Sel input do ?
Answer: the Op2Sel input chooses whether the ALU operation selected (for instance MOV or ADD) will be performed on a register or on an immediate value. If Op2Sel = 0, then the data for the operation comes from a register, for example Ra := Rb. On the other hand, if Op2Sel = 1, then the data for the operation comes from an immediate value, for instance Ra := #14.
Remark: Op2Sel input corresponds to the 8th bit of the current instruction, known as ISM(8) (see Figure 4).

Now to make the same truth table for the ADD instruction.

A	B	C	Imm8	Op2Sel	AD1SelC	ALUOPC	Regfile.Din1
a	b	c	X	0	0	1 (ADD)	Ra := Rc + Rb
a	b	c	X	0	1	1 (ADD)	Rc := Ra + Rb
a	X	X	k	1	0	1 (ADD)	Ra := Ra + k
X	X	c	k	1	1	1 (ADD)	Rc := Rc + k

In this case, the instructions which set R_C to be changed instead of Ra are again invalid. Therefore the two rows which correspond to the **correct dataflows are rows 2 and 3** ! (for **ADD** instruction, one of the correct dataflow rows has an AD1SelC at a 1, when it was always 0 for **MOV** instruction.

ALUOPC	INS(8)	Op2Sel	AD1SelC
0 (MOV)	0	0	0
0 (MOV)	1	1	0
1 (ADD)	0	0	1
1 (ADD)	1	1	0

Question: Are INS(8) and Op2Sel the same? If so, then why have two separate columns for them?
Attempt at answer (may be incorrect): from my current understanding, INS(8) (the 8th bit of the instruction) and Op2Sel are the same for this case, when we are only interested in the MOV and ADD instructions.
My guess as to why we need to have both of them separately, even though they are identical for the purposes of this truth table, is that we need INS(8) to be an input and Op2Sel to be an output because this truth table is for a subsheet (dpdecode I believe) - so the inputs and outputs are fixed.
Answer: Op2Sel and INS(8) are indeed functionally the same thing.

Hence, with this understanding, we can make the complete truth table:

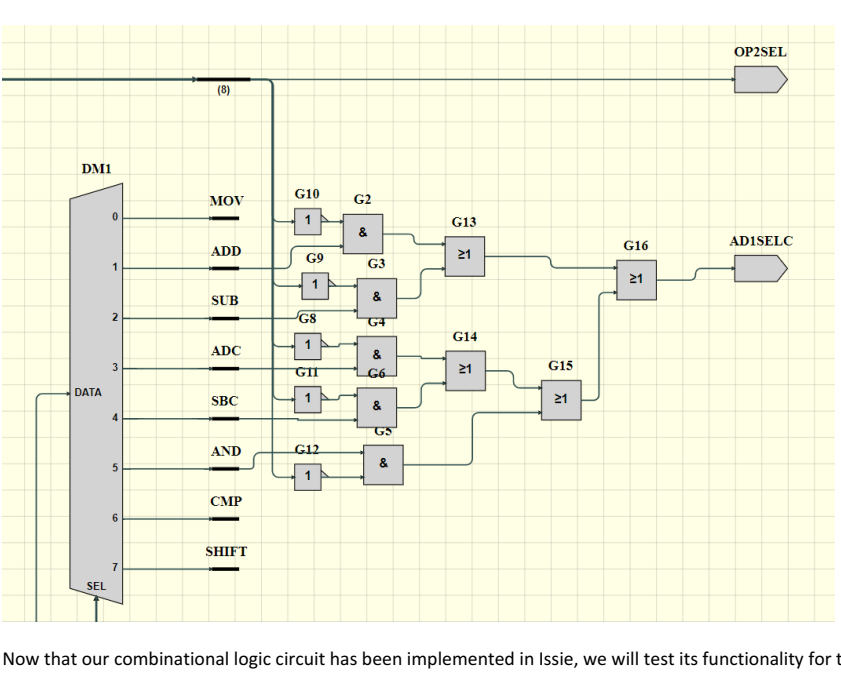
Instruction	ALUOPC	INS(8)	Op2Sel	AD1SelC
`MOV Ra, Rb`	0	0	0	0
`MOV Ra, #IMM`	0	1	1	0
`ADD Rc, Ra, Rb`	1	0	0	1
`ADD Ra, #IMM`	1	1	1	0
`SUB Rc, Ra, Rb`	2	0	0	1
`SUB Ra, #IMM`	2	1	1	0
`ADC Rc, Ra, Rb`	3	0	0	1
`ADC Ra, #IMM`	3	1	1	0
`SBC Rc, Ra, Rb`	4	0	0	1
`SBC Ra, #IMM`	4	1	1	0
`AND Rc, Ra, Rb`	5	0	0	1
`AND Ra, #IMM`	5	1	1	0
`CMP Ra, Rb`	6	0	0	(0)
`CMP Ra, #IMM`	6	1	1	(0)

Task 4 complete.

Task 5 - Using your completed truth table, implement correct combinational logic in the dpdecode block to drive the dpdecode.AD1Sel and dpdecode.Op2Sel outputs. Test your logic for ADD and MOV instructions by setting the instruction ROM in the Lab1 and eep1Model Issie projects to source data from lab1testmovadd.ram file. Compare the REGFILE output waveform changes in the two simulations. The next sections will test the other ALU instructions.

Interpretation: we need combinational logic to drive the AD1SelC and Op2Sel outputs of the dpdecode block.
We will implement this circuit in Issie, in the lab1.dprj project.
Once this step is done, we will test our combinational logic for **ADD** and **MOV** instructions.
We will also run a waveform simulation in Issie to be able to observe the changes.

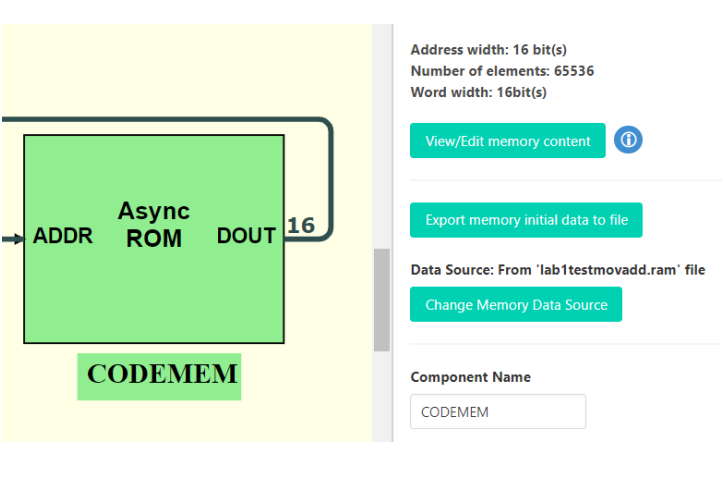
Here is my combinational logic, implemented in Issie:



Now that our combinational logic circuit has been implemented in Issie, we will test its functionality for the **MOV** and **ADD** instructions.
In order to do so, the instructions for Task 5 tell us to do the following:

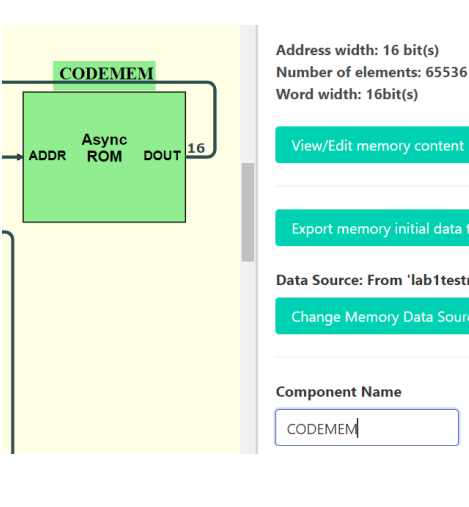
a) Set the instruction ROM in the lab1 and eep1model Issie projects to the source data from lab1testmovadd.ram file.
b) Compare the REGFILE output waveform changes in the 2 simulations
c) If they are the same, then we can say that our combinational logic is working correctly

In the lab1 project, the ROM is indeed set to the correct file:



Address width: 16 bit(s)
Number of elements: 65536
Word width: 16bit(s)
Data Source: From 'lab1testmovadd.ram' file
Component Name: CODEMEM

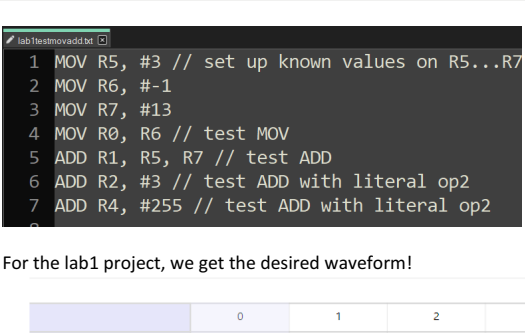
This is also the case in the eep1model project:



Address width: 16 bit(s)
Number of elements: 65536
Word width: 16bit(s)
Data Source: From 'lab1testmovadd.ram' file
Component Name: CODEMEM

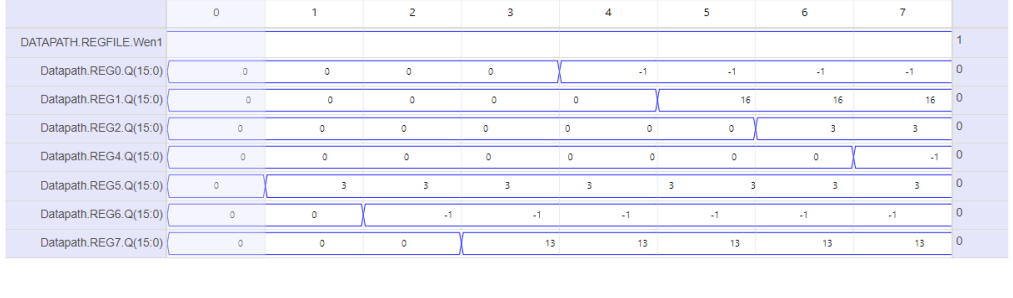
Here is what we expect to see for both waveforms: (table made by hand using the .txt assembly code)

	0	1	2	3	4	5	6	7
Reg 0	0	0	0	0	-1	-1	-1	-1
Reg 1	0	0	0	0	0	16	16	16
Reg 2	0	0	0	0	0	0	3	3
Reg 4	0	0	0	0	0	0	0	255
Reg 5	0	3	3	3	3	3	3	3
Reg 6	0	0	-1	-1	-1	-1	-1	-1
Reg 7	0	0	0	13	13	13	13	13

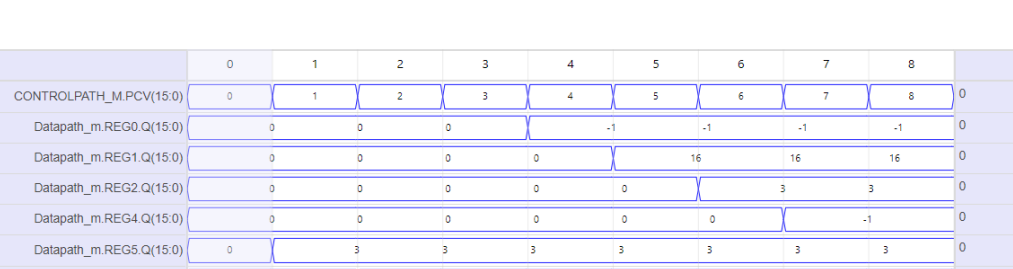


```
MOV R5, #3 // set up known values on R5...R7
MOV R6, #-1
MOV R7, #13
MOV R0, R6 // test MOV
ADD R1, R5, R7 // test ADD
ADD R2, #3 // test ADD with literal op2
ADD R4, #255 // test ADD with literal op2
```

For the lab1 project, we get the desired waveform!



And also for thee eep1model!



Since the 2 waveforms match, we can confirm that the combinational logic we designed is working correctly for the **MOV** and **ADD** instructions.
The only difference between the table I made by hand and the final waveforms is that I predicted Reg4 would store a value of 255 by the end of the program, whereas the waveforms show it actually stores a value of -1. This is because $255_{10} = 1111\ 1111_2$ which, in signed binary 2s complement, is equal to -1. (I am viewing the waveforms in Issie in the signed decimal mode for easier readability).

Task 5 complete.

Task 6 - Implement the ALUDECODE logic using MUX1, MUX2 (which makes this implementation very easy). Note that the ALUDECODE outputs are don't care for all the ALU instructions that do not use the ADDSUB block. Implement logic to drive dpdecode.WEN1. Test that your design matches the model design for all the instructions in lab1testarith.txt by changing instruction ROM sources (Issie properties) to lab1testarith.ram and running simulations.

Question: What is the ALUDECODE.ADDSUBCIN output ?
Attempt at answer: ADDSUBCIN is the output of the ALUDECODE block, which then is used as the Carry in input to the ADDSUB block in the ALU. In order to perform subtraction, the ALU needs to first invert the bits of the subtrahend, then add 1, and finally add the minuend and the 2s complement of the subtrahend. Therefore, for a subtraction, the carry in needs to be set to 1, hence ALUDECODE.ADDSUBCIN = 1 for subtraction ALU instructions.

INA	INB	CARRYIN	INVERT	ADDSUB.OUT
a	b	0	0	a + b
a	b	1	0	a + b + 1
a	b	0	1	a + ¬b
a	b	1	1	a - b

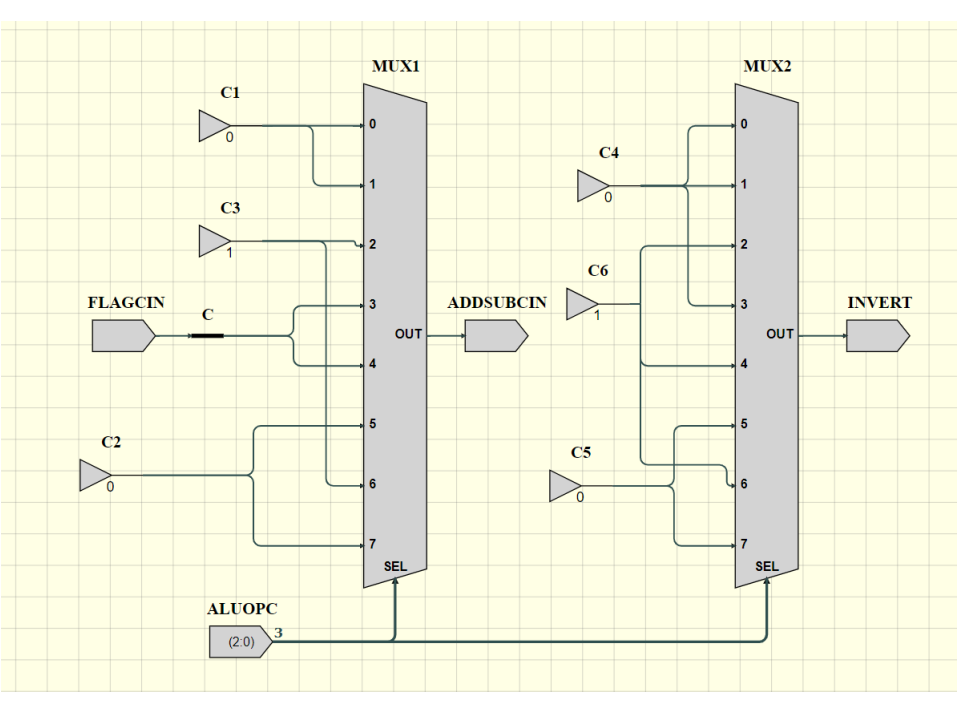
Here is the full truth table for the ALUDECODE logic.

INSTRUCTION	ALUOPC	FlagC	ADDSUBCIN	INVERT	W FLAGC	WEN1
`MOV`	0	C	0	0	0	1
`ADD`	1	C	0	0	1	1
`SUB`	2	C	1	1	1	1
`ADC`	3	C	C	0	1	1
`SBC`	4	C	C	1	1	1
`AND`	5	C	0	0	0	1
`CMP`	6	C	1	1	1	0

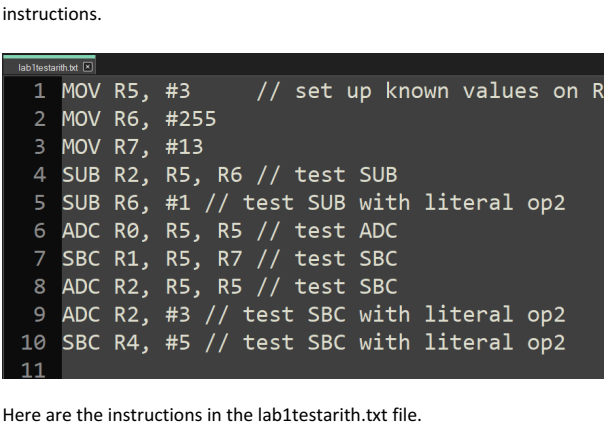
Remark: FLAGC is the carry from a previous operation, stored in a flip-flop.
ADDSUBCIN is the carry bit that gets passed through into the ADDSUB block of the ALU.
WFLAGC is the enable for writing FLAGC. We want this to be on if the carry of the current ALU operation has significance. For example, if we are doing an ADD or a SUB operation, the carry is useful to store, hence the enable to writing FlagC will be on, and the carry from the operation will be saved. On the other hand, if, for instance, we are executing a MOV instruction, then the carry bit does not have any meaning, and therefore we will not save it by setting WFLAGC to 0.

Remark: WEN1 output simply determines whether the output of the ALU operation will be written to a register or not, so it is on for all ALU instructions except CMP.

Here is the combinational logic that implements the truth table seen above.



Now that our combinational logic is complete within Issie, we can test its behaviour. In order to do so, we will import the lab1testarith.ram code into the ROM for our lab1 dprj project. We will also do the same in the eep1model.dprj project. We will then be able to compare the two waveforms to determine if our combinational logic is working correctly for these ALU instructions.



```
MOV R5, #3     // set up known values on R
MOV R6, #255
MOV R7, #13
SUB R2, R5, R6 // test SUB
SUB R6, #1 // test SUB with literal op2
ADC R0, R5, R5 // test ADC
SBC R1, R5, R7 // test SBC
ADC R2, R5, R5 // test SBC
ADC R2, #3 // test SBC with literal op2
SBC R4, #5 // test SBC with literal op2
```

Here are the instructions in the lab1testarith.txt file.
Calculating by hand, this is what we are expecting to see:

	1	2	3	4	5	6	7	8	9	10
R0	0	0	0	0	0	7	7	7	7	7
R1	0	0	0	0	0	0	-11	-11	-11	-11
R2	0	0	0	4	4	4	4	6	9	9
R4	0	0	0	0	0	0	0	0	0	-6
R5	3	3	3	3	3	3	3	3	3	3
R6	0	-1	-1	-1	-2	-2	-2	-2	-2	-2
R7	0	0	13	13	13	13	13	13	13	13

* At the 6th instruction, we have ADC R0, R5, R5. The value of R5 + R5 is 6, however the result of the operation is 7 because we used ADC and not ADD, and the carry from the previous operation was 1 because:

Instruction 5 : SUB R6, #255, #1

$$-1_{10} - 1_{10} = (1111\ 1111)_{s2} - (0000\ 0001)_{s2}$$
$$= (1111\ 1111)_{s2} + (1111\ 1110)_{s2} + 1_2$$
$$= \boxed{1}1111\ 1110$$

here is the carry → THESE ARE IMMEDIATE VALUES, SO 8-BITS ! THAT IS WHY WE GET A CARRY.

7: SBC R1, R5, R7 ⇒ R1 := 3 − 13 + C − 1

Previous instruction was: ADC R2, R5, R5

⇒ R2 := R5 + R5

```
  0000 0011
+ 0000 0011
+ 0000 0001
  0000 0111
```

As we can see, this operation does not produce a carry.

Moreover, the W FLAGC EN is on for an ADC instruction, so we know that FLAGC now equals 0.

Therefore, our SBC instruction is:

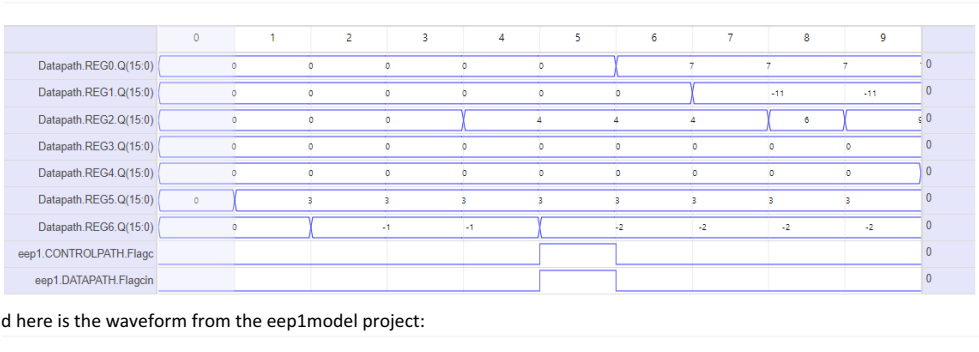
R1 := 3 − 13 + 0 − 1 = −11

7th instruction : 3 − 13 + 0 − 1

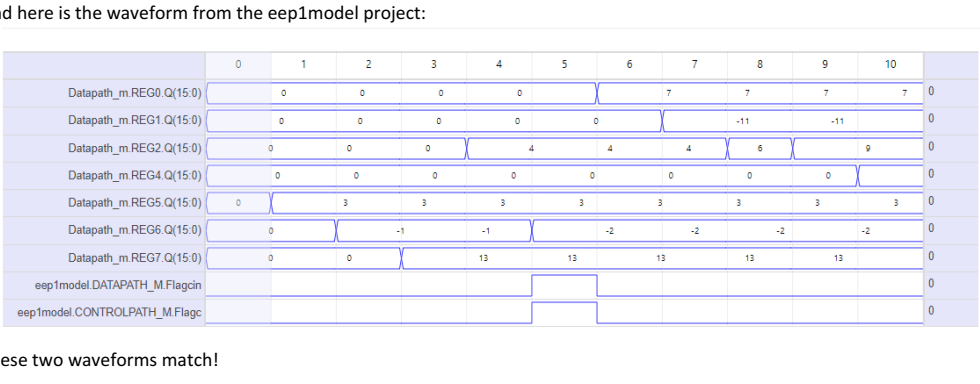
```
⇒   0 0 0 0  0 0 1 1
  − 0 0 0 0  1 1 0 1
  − 0 0 0 0  0 0 0 1

⇒   0 0 0 0  0 0 1 1
  + 1 1 1 1  0 0 1 1   (2s complement)
  + 1 1 1 1  1 1 1 1   (2s complement)

= ① 1 1 1 1  0 1 0 1
```

carry? NO! WE ARE IN 16 BITS !!!!!!
Such a silly mistake.

This is the waveform from the lab1 Issie project:



And here is the waveform from the eep1model project:



These two waveforms match!

Task 6 complete.

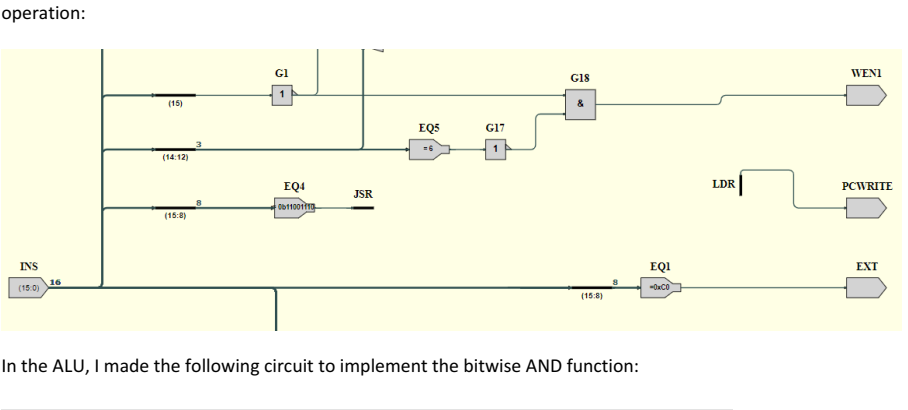
Task 7 - Look at the logic that drives DPDECODE.WEN1. Check (write a truth table if you like) what is the value of WEN1 for each of the 8 ALU instructions. Compare this with the ALU instruction specifications and identify where it is incorrect. Look at the logic in ALU and identify how to connect an Issie "N bits AND" component to implement the AND instruction correctly. Alter the logic in ALU and dpdecode sheets to implement AND and CMP instructions. Test that your design matches the model design for all the instructions in lab1testandcmp.txt.

By looking at the dpdecode sheet in the lab1project, we can see that, as it stands, the logic that drives WEN1 is the following:
WEN1 = ¬ INS(15).

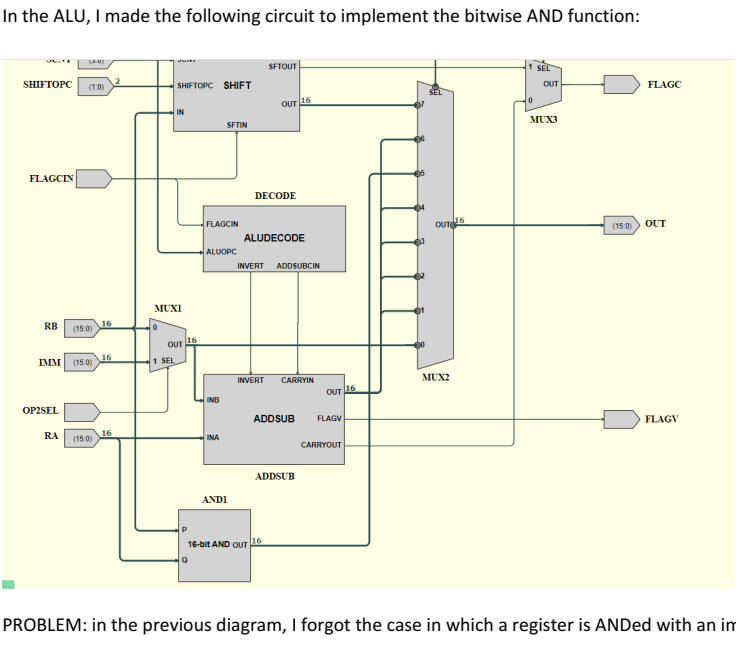
Here is the truth table which describes the relationship between WEN1 and ALUOPC:

ALUOPC	Operation	WEN1
0	MOV	1
1	ADD	1
2	SUB	1
3	ADC	1
4	SBC	1
5	AND	1
6	CMP	0
7	Shift	1

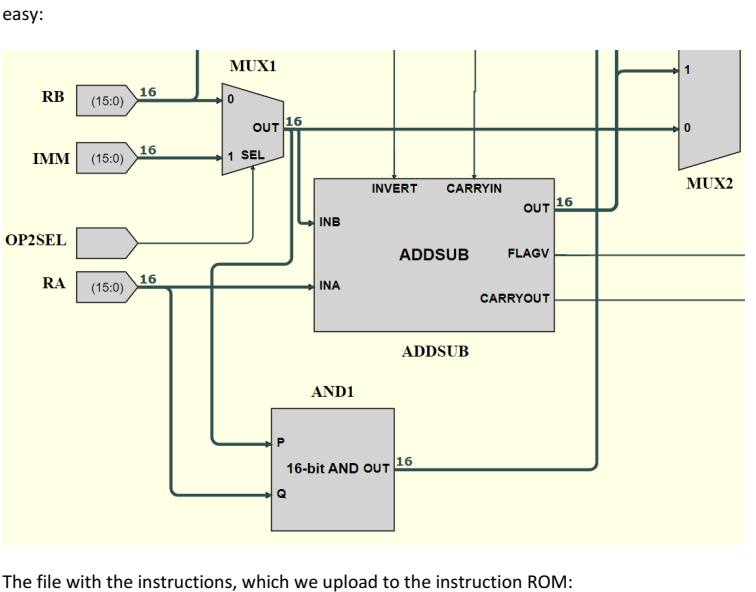
Clearly the logic present by default in the lab1 project is not the correct combinational logic to drive the WEN1 output.
ALUOPC $6_{10} = 110_2$ --> we want WEN1 to be 0 when the 3 instruction bits which specify ALUOPC are 110, and 1 otherwise.
The following circuit, implemented in dpdecode, checks if ALUOPC (bits 12 through 14 of the instruction word) is equal to 6. The signal then gets inverted, and passed through to WENA1.
Update: the previous condition needs to be true as described, and, additionally, INS(15) needs to be 0, since it is an ALU operation:



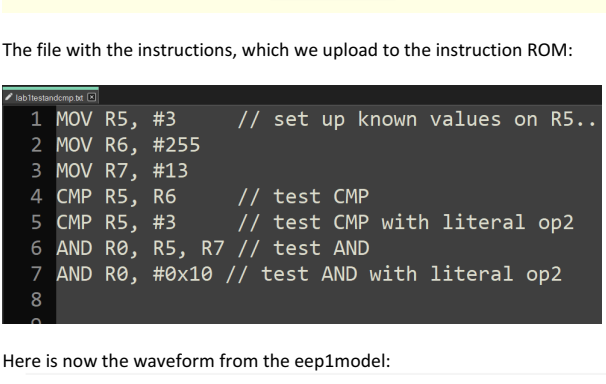
In the ALU, I made the following circuit to implement the bitwise AND function:



PROBLEM: in the previous diagram, I forgot the case in which a register is ANDed with an immediate value. Thankfully, the fix is easy:

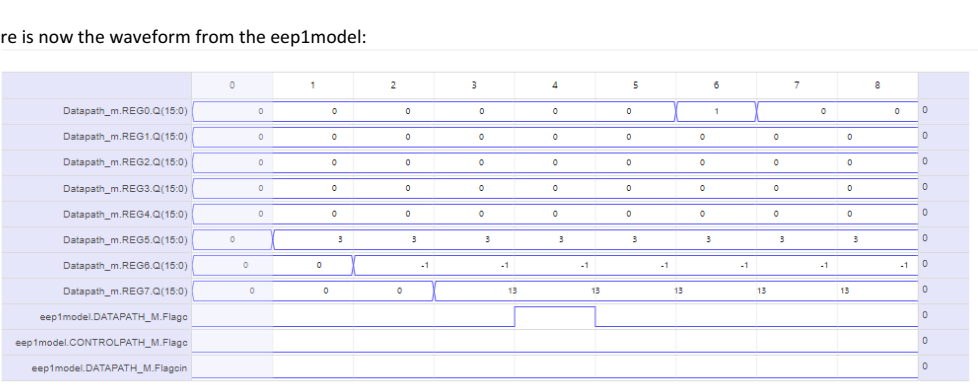


The file with the instructions, which we upload to the instruction ROM:

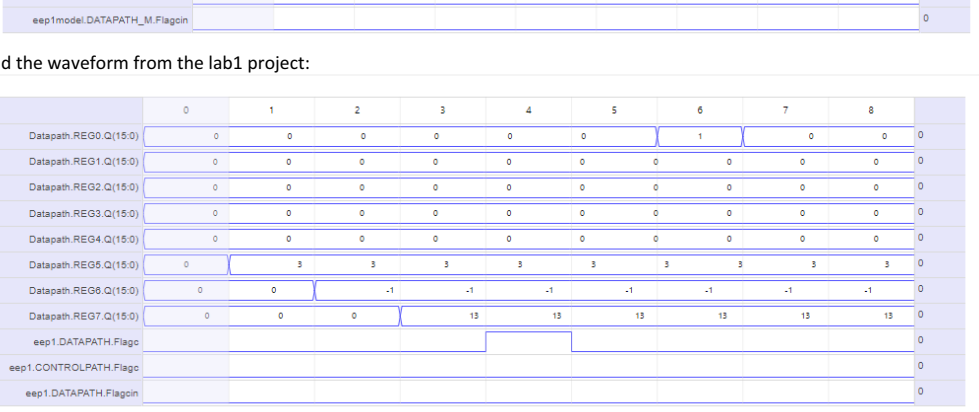


```
MOV R5, #3     // set up known values on R5..
MOV R6, #255
MOV R7, #13
CMP R5, R6     // test CMP
CMP R5, #3     // test CMP with literal op2
AND R0, R5, R7 // test AND
AND R0, #0x10 // test AND with literal op2
```

Here is now the waveform from the eep1model:



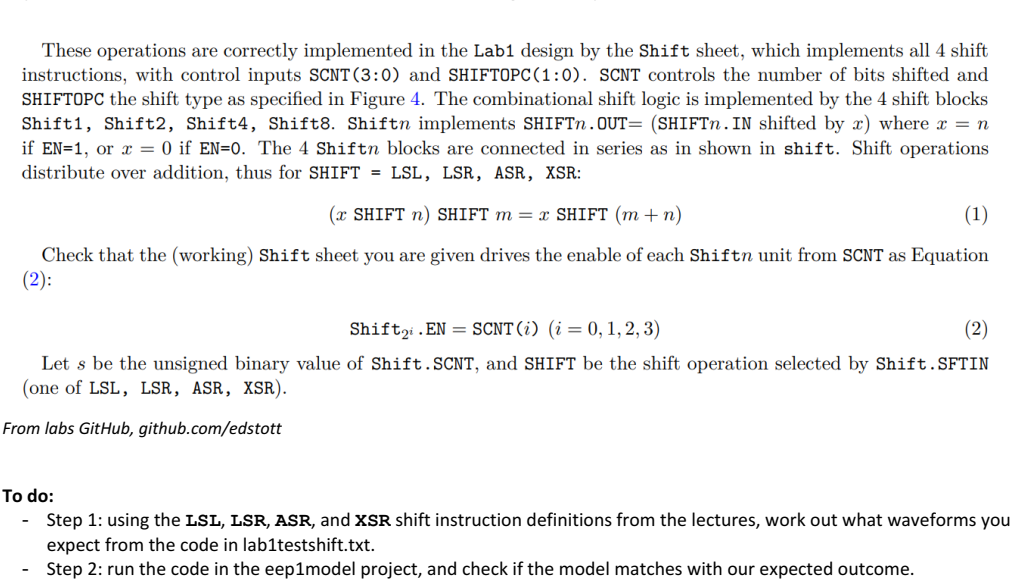
And the waveform from the lab1 project:



They match! Which means our model is correct.

Task 7 complete.

Task 8 - Using the LSL,LSR,ASR,XSR shift instruction definitions from the lectures work out what waveforms you expect from the code in lab1testshift.txt . Check this against eep1model.



These operations are correctly implemented in the Lab1 design by the Shift sheet, which implements all 4 shift instructions, with control inputs SCNT(3:0) and SHIFTOPC(1:0). SCNT controls the number of bits shifted and SHIFTOPC the shift type as specified in Figure 4. The combinational shift logic is implemented by the 4 shift blocks Shift1, Shift2, Shift4, Shift8. Shift$n$ implements Shift$n$.OUT= (Shift$n$.IN shifted by x) where $x = n$ if EN=1, or $x = 0$ if EN=0. The 4 Shift$n$ blocks are connected in series as in shown in shift. Shift operations distribute over addition, thus for SHIFT = LSL, LSR, ASR, XSR:

$$(x \text{ SHIFT } n) \text{ SHIFT } m = x \text{ SHIFT } (m+n) \qquad (1)$$

Check that the (working) Shift sheet you are given drives the enable of each Shift$n$ unit from SCNT as Equation (2):

$$\text{Shift}_{2^i}.\text{EN} = \text{SCNT}(i)\ (i = 0, 1, 2, 3) \qquad (2)$$

Let s be the unsigned binary value of Shift.SCNT, and SHIFT be the shift operation selected by Shift.SFTIN (one of LSL, LSR, ASR, XSR).

From labs GitHub, github.com/edstott

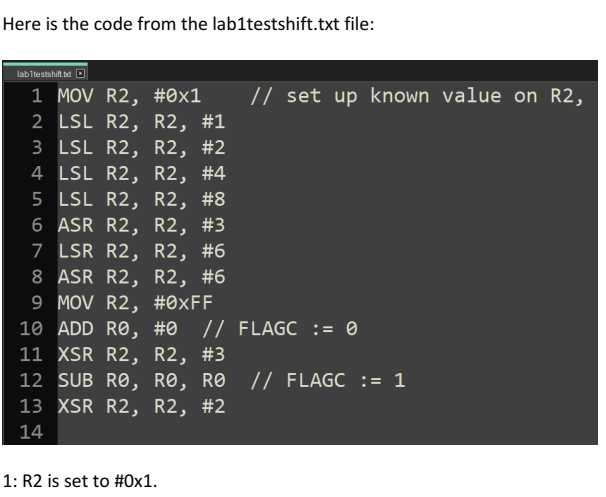
To do:

- Step 1: using the **LSL, LSR, ASR,** and **XSR** shift instruction definitions from the lectures, work out what waveforms you expect from the code in lab1testshift.txt.
- Step 2: run the code in the eep1model project, and check if the model matches with our expected outcome.
- Step 3: Check that the (working) "Shift" sheet we are given drives the enable of each Shiftn unit from SCNT as (E_2).

Step 1:

Mnemonic	Meaning	Operation
LSL	Logical shift left	×2^n (unsigned)
LSR	Logical shift right	÷ 2^n (unsigned)
ASR	Arithmetic shift right	÷ 2^n (signed)
XSR	Extended shift right	multi-word 1-bit shift right

Here is the code from the lab1testshift.txt file:



```
 1 MOV R2, #0x1    // set up known value on R2,
 2 LSL R2, R2, #1
 3 LSL R2, R2, #2
 4 LSL R2, R2, #4
 5 LSL R2, R2, #8
 6 ASR R2, R2, #3
 7 LSR R2, R2, #6
 8 ASR R2, R2, #6
 9 MOV R2, #0xFF
10 ADD R0, #0   // FLAGC := 0
11 XSR R2, R2, #3
12 SUB R0, R0, R0  // FLAGC := 1
13 XSR R2, R2, #2
```

1: R2 is set to #0x1.
2: R2 is shifted Left by 1 bit, equivalent to multiplying by 2.
3: R2 is shifted Left by 2 bits, equivalent to multiplying by 4.
4: R2 is shifted Left by 4 bits, equivalent to multiplying by 16.
5: R2 is shifted Left by 8 bits, equivalent to multiplying by 256.
6: R2 is arithmetically shifted Right by 3 bits, equivalent to dividing by 8.
And so on...

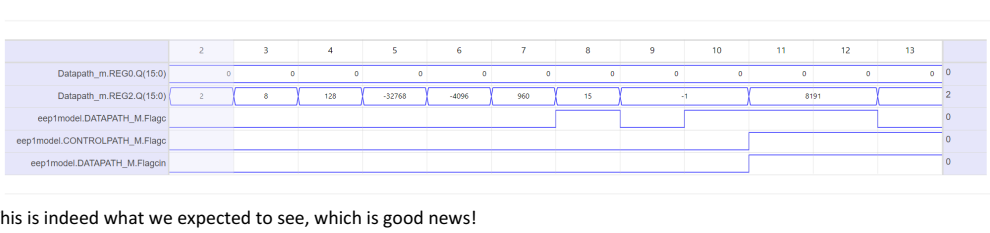
	R0	R2	FlagC
1 - `MOV`	X	0000 0000 0000 0001 = 1	0
2 - `LSL`	X	0000 0000 0000 0010 = 2	0
3 - `LSL`	X	0000 0000 0000 1000 = 8	0
4 - `LSL`	X	0000 0000 1000 0000 = 128	0
5 - `LSL`	X	1000 0000 0000 0000 = -32678	0
6 - `ASR`	X	1111 0000 0000 0000 = -4096	0
7 - `LSR`	X	0000 0011 1100 0000 = 960	0
8 - `ASR`	X	0000 0000 0000 1111 = 15	0
9 - `MOV`	X	1111 1111 1111 1111 = -1	0
10 - `ADD`	0	X	0
11 - `XSR`	0	0001 1111 1111 1111 = 8191	1
12 - `SUB`	0*	X	1
13 - `XSR`	X	0000 0111 1111 1111 = 2047	0

12: SUB R0 := R0, R0 -->

```
  0000 0000 0000 0000
+ 1111 1111 1111 1111
+ 0000 0000 0000 0001
= 1 0000 0000 0000 0000
```

--> 0, but with a carry of 1.

Step 2:
Here is the waveform we get from the eep1model:



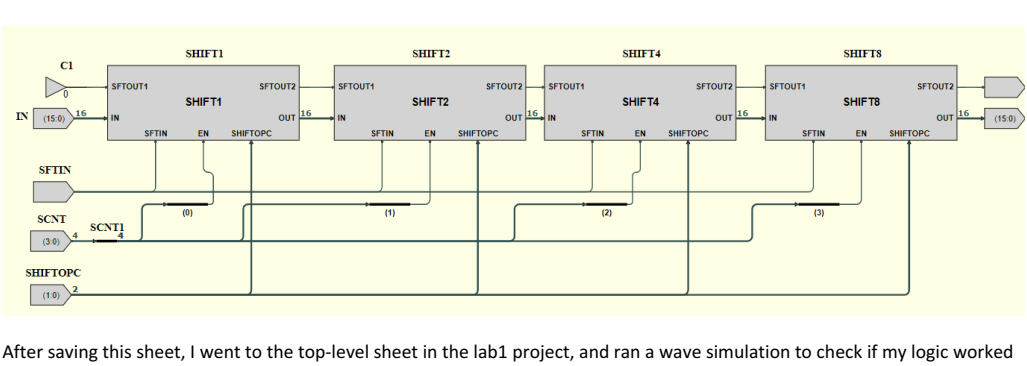
This is indeed what we expected to see, which is good news!

Step 3:

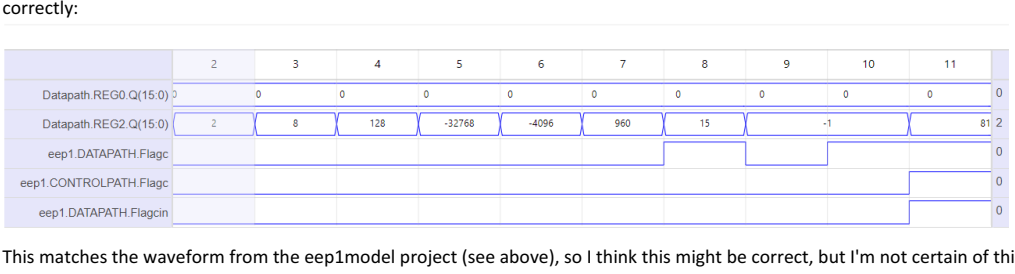
We need to check that the (working) "Shift" sheet we are given drives the enable of each Shiftn unit from SCNT as (E_2).

(E_2): Shift_{2^i}. EN = SCNT(i), (i = 0,1,2,3)

Problem: my "shift" sheet from the lab1 project was not connected properly; I had to connect the enables for the different shift units myself. Here is the logic I used, implemented in Issie:



After saving this sheet, I went to the top-level sheet in the lab1 project, and ran a wave simulation to check if my logic worked correctly:



This matches the waveform from the eep1model project (see above), so I think this might be correct, but I'm not certain of this step.